高流量淺談
以面對搶票為例

by

Triton Ho

前言

今天目的是教學
不是求職面試
所以不採用 *top-down approach*

定義「搶票」

- 在網站開賣超熱門商品，該商品
  - 有限定數量（通常 *1000* 內）
  - 絕對不能 *over-sell*
  - 有過萬人想買到該商品，而且大家都會等開售時立即按下購買
- 例子*:*
  - 某明星簽名的限量球鞋
  - 中國的「搶紅包」活動

面對「搶票」

- 盡量不要讓 *bot* 拿走太多商品
- 讓系統不要整個掛掉 *blank page*
- 別讓用戶把資料全填好後才拿不到票

如果廁所是「稀有資源」.....

- 不是大小便的不準進廁所
- 進廁所前請預先鬆開褲子
  進了廁格後就趕快拉出來
- 嚴禁在廁所內玩手機 / 看報紙
  拉好了就立即穿好衣物出來
- 盡可能讓使用者加速拉屎拉尿

orm

今天都 *2026* 了

- 完全不用 *orm* 的，你是活在二疊紀的（放心，恐龍還有三疊紀和侏羅紀才滅亡）
- *100%* 只用 *orm* 的，你沒經歷過真正複雜 / 高流量的系統

Optimistic lock in ORM

- 這是 *orm* 的標準功能，用來防止 *race condition*
- 例子：
  - *Get record from Database*
  - *var original_ts = record.update_ts*
  - *Update the record content*
  - *Update record*
    Set columnA = blablabla,
    update_ts = now()
    where id = $id and update_ts = $original_ts

Optimistic lock 優點

- 優點
  - 在絕大多數場景下
    Optimistic lock 能提供足夠保護力
  - *Optimistic lock* 沒有真正的用上 *lock*
    所以不會引起 *deadlock* 問題
  - 對新手超級友善
    只要叫 *orm* 打開 *optimistic lock* ，幾乎沒有犯錯可能
    （謎之聲：新手就是連 *copy-cat* 都可以錯的..... ）

Optimistic lock 缺點

- 一旦資料在中間被人改動過：
  - 要麼就是把資料再拿一次，再重新跑一次 *business logic*（就是 *git pull –rebase* ）
  - 要麼就是 *return error* 叫用戶重新 *resubmit request*
- 一般而言會是後者
  因為 *exception* 不一定能被 *handle* 的
  （而且如果 *race condition* 不常發生下，不值得花時間這樣子做）

搶票時的 *Optimistic lock*

- 如果 *1000* 人同時按下購買.....
  - 只有一個會 *success* ，另外 *999* 人會 *rollback* 然後重來
- 你的系統會被大量「無效」的 *transaction* 「佔領」，然後不管是否在搶票，都拿到 *blank page*
  - 因為 *DB connection* 被佔光光了

Framework 和 *orm*

- 如果你在用像 *Spring boot* 之類的 *framework*
  - 收到 *request* 時自動 *begin db transaction*
    response 時自動視有沒有 *error* 來 *commit / rollback*
- 這一般是沒問題的，但是.....
  你購買流程，正常要跟 *Stripe / Paypal* 之類金流系統通訊吧

別把第三方系統行動包在 *TX* 內

- 你家公司沒有跟金流系統結拜好兄弟
  - 他們系統死了，別被拉扯一起死好嗎（台灣叫抓交替？）
- 最要命的不是對方系統掛了
  而是卡住 *10* 秒才回應
- 即使對方系統正常， *response time* 就當 *30ms* 好了

  這樣子，熱門商品的 *product object* 就會被你的 *TX* 佔用額外 *30ms*

理想的流程

- *Begin DB transaction*
  - *//檢查用戶是否合資格購買商品*
  - *Insert into purchase_order*
    with payment_id = null
  - *Update product set remaining_stock -= 1*
    where id = ? and remaining_stock >= 1
  - *If AffectedRowCount() == 0*
    return SoldOutError and Rollback
- *Commit DB transaction*
- *Create payment object in Payment Gateway, and wait for payment_id*
- *Begin DB transaction*
  - *Update purchase_order*
    Set payment_id = ?payment_id
- *Commit DB transaction*

Database schema

- 新手/走火入魔的中手
  - *Product*
    (without remaining_count column)
- 中手
  - *Product*
    (with remaining count column)
- 老手
  - *Product*
    (without remaining_count column)
  - *Product_stock*
    (product_id, total_count, remaining_count)
    PK: product_id, FK to Product

Database tuning

- 把 *product_stock* 的 *table fill_factor* 改低
  （ *postgresql default* 是 *90%* ）
- 用戶購買東西的 *endpoint* ，改用獨立 *db connection pool*
- 理想的 *db connection pool size* ：
  *CPU_thread * (0.25 to 0.5)*
  絕對不是 *100*
- 「 *PHP* 沒有 *connection pool* 也好，是因為 *PHP* 不需要 *connection pool* 」
  有一種外星人叫「 *PHP / MySQL* 無敵好棒棒 」.....
  - *Pgpool 2 / proxySQL* 也能做到 *connection pool* 的.....

ratelimiting

- *Read-only request* 請善用 *CDN*
- 首先， *db connection pool / load balancer* 要設定好
  - 不然，就會有１０萬個 *user* 同時連上 *main DB*
    （然後你的 *main DB* 掛掉.....）
- *Per-IP limiting* 請找 *cloudflare* 之類的供應商，別自己來
- （在正確使用下）
  如果你的 *ecom* 網站付不起 *cloudflare / CDN* ，你家還是收一收好了.....

Ratelimiting is hard

- *Token bucket algorithm* 只需要一個 *floating point value in Redis*
  - 其 *implementation* 大約是 *<=20* 行 *lua script* 吧
- 如果每一個 *request* 都要一個 *redis operation* 搶票時你的系統很很可能出事
  - 理論上 *redis node* 能接受 *200K req per second*
  - 現實上， *io* 讓 *redis* 沒法做到 *200K req per second*

Ratelimiting with local buffering

- (收費內容，請付石虎費 ^^)

Ratelimiting 碎碎念

- 請把 *ratelimiting* 放在 *load balancing layer*
- 如果在 *AS layer* 來做，請務必放在 *begin DB transaction* 之前
- 再次強調：搶票的瞬間流量可以沒有上限的

淺談 *bot*

- *Bot* 最大問題：
  能用 *bot* 的，很可能都是專業的倒賣炒家
  你的熱門商品都最終都落入倒賣炒家上
- 作為一般用戶：
  - 他總是搶不到商品，然後看到商品總是在網上被高價倒賣.....
  - 用戶想法：
    喵的，網站都是賣假的，一定是有員工私下倒賣！

淺談防範 *bot*

- 如果要即時分辨一個 *request* 是來自真人還是 *bot……*
  - 我不知道答案，請找 *cloudflare* 員工來答你 *XD*
- 但是，從之前熱門商品 *application log* ，要分辨誰是 *bot* 不是太難的
  - 這不需要 *AI* ，簡單 *data analysis* 都能做到吧
- 在開賣熱門商品之前，我們可以替每一個用戶都建立一個 *bot_score* ，越高分數就越有可能是 *bot*

防範 *bot* 實作

- （收費內容，請付石虎費 ^^ ）

淺談 *Event Sourcing*

- 請參見
  - *https://learn.microsoft.com/en-us/azure/architecture/patterns/event-sourcing*
- 重點
  - *Immutable event object, append-only*
  - 一般是 *Eventual-consistency*
  - 系統 *Current state =* 從 *Day 1 replay* 所有的 *event*
  - 為了效能，一般會配上 *CQRS*
    - 簡單而言： *materialized view*

Event Sourcing 實用例子

- *Stock Exchange*
  - =.=

完